



ICT2216/ICT2516C

Secure Software Development

Secure Software Concepts

Raymond Chan & Tram Truong Huu

Agenda

- Relationship between Software and Security
- Cases studies
- Software Vulnerabilities
 - OWASP Top 10
- Vulnerability Tracking
 - CVEs
 - NVD
 - CAPEC
 - MITRE ATT&CK
- Software Assurance Methodologies
 - BSIMM
 - OpenSAMM
 - Microsoft SDLC

VULNERABILITY TRACKING

What if...

- As a software developer, how can I learn more about an attack? So that I can do the proper patching?
- As a security engineer, where I can retrieve the day-one attack information (why not zero-day)? So that I can prevent the attack from happening to my system?



Vulnerability tracking

How can you find the latest vulnerability and learn types of attacks?

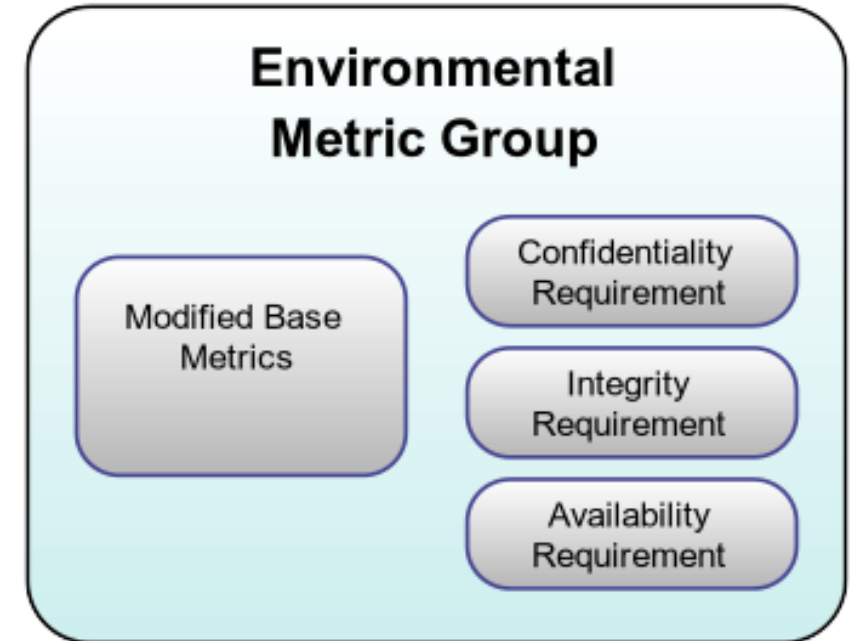
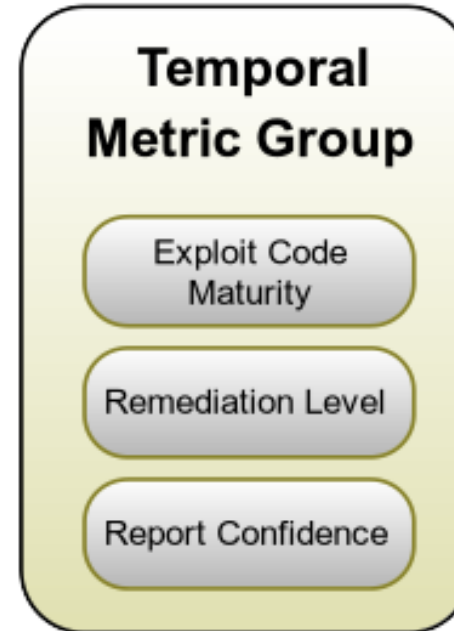
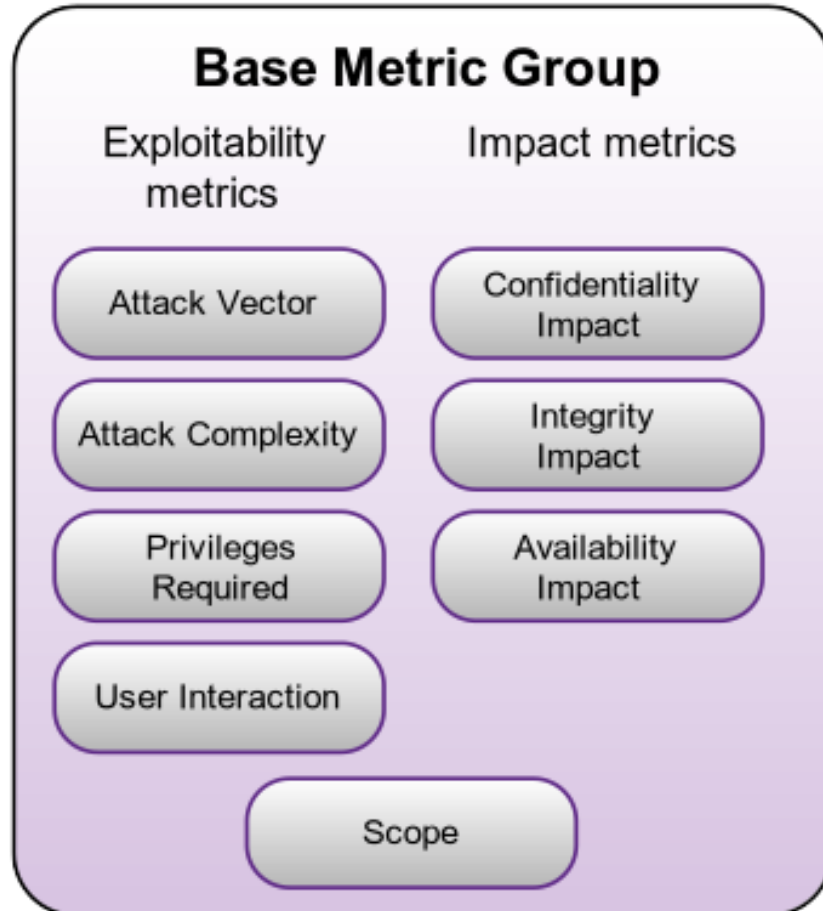
Subscribe the following:

- Common Vulnerabilities and Exposures (CVE)
- National Vulnerability Database (NVD)

Understand the attacks:

- Common Attack Pattern Enumeration and Classification (CAPEC)
- MITRE ATT&CK

Common Vulnerability Scoring System - CVSS



Rating	CVSS Score
None	0.0
Low	0.1 - 3.9
Medium	4.0 - 6.9
High	7.0 - 8.9
Critical	9.0 - 10.0

Common Vulnerabilities and Exposures

(CVE): <https://cve.org/>

- A directory of publicly known information security vulnerabilities and exposures
- You can submit findings to CVE!



Common Vulnerabilities and Exposures

 [About](#) [Partner Information](#) [Program Organization](#) [Downloads](#) [Resources & Support](#) [Report/Request](#)

[Search tips](#) | [Provide feedback](#)

Notice: Expanded keyword searching of CVE Records ([with limitations](#)) is now available in the search box above. [Learn more](#) .

CVE™ Program Mission

Identify, define, and catalog publicly disclosed cybersecurity vulnerabilities.

There are currently over **330,000** CVE Records accessible via [Download](#) or **Keyword Search** above.

The CVE Program partners with community members worldwide to grow CVE content and expand its usage. Click below to learn more about the role of **CVE Numbering Authorities (CNAs)** and **Roots**.

[Learn More](#)

[Become a Partner](#)

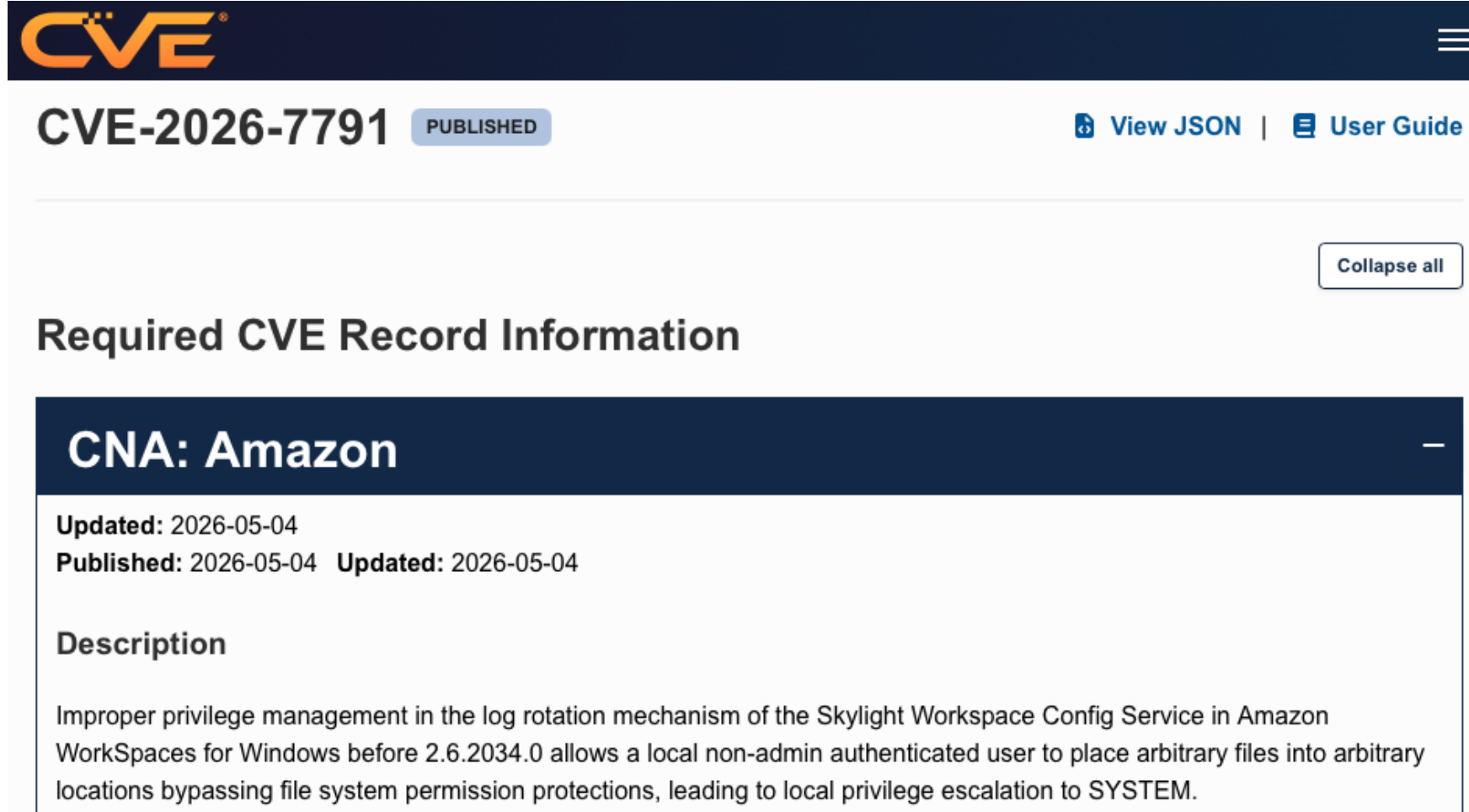


News

-  [OpenAI Added as CVE Numbering Authority \(CNA\)](#)
-  [Cloud Security Alliance Added as CVE Numbering Authority \(CNA\)](#)

CVE Example

- <https://www.cve.org/CVERecord?id=CVE-2026-7791>



The screenshot shows the CVE Record page for CVE-2026-7791. The page has a dark blue header with the CVE logo and a hamburger menu. Below the header, the CVE ID "CVE-2026-7791" is displayed in large text, followed by a "PUBLISHED" status badge. To the right of the ID are links for "View JSON" and "User Guide". A "Collapse all" button is located on the right side of the page. The main content area is titled "Required CVE Record Information" and contains a section for "CNA: Amazon". Under this section, the "Updated" date is listed as "2026-05-04", and the "Published" date is also listed as "2026-05-04". The "Description" section follows, containing the text: "Improper privilege management in the log rotation mechanism of the Skylight Workspace Config Service in Amazon WorkSpaces for Windows before 2.6.2034.0 allows a local non-admin authenticated user to place arbitrary files into arbitrary locations bypassing file system permission protections, leading to local privilege escalation to SYSTEM."

CVE-2026-7791 PUBLISHED [View JSON](#) | [User Guide](#)

[Collapse all](#)

Required CVE Record Information

CNA: Amazon

Updated: 2026-05-04
Published: 2026-05-04 **Updated:** 2026-05-04

Description

Improper privilege management in the log rotation mechanism of the Skylight Workspace Config Service in Amazon WorkSpaces for Windows before 2.6.2034.0 allows a local non-admin authenticated user to place arbitrary files into arbitrary locations bypassing file system permission protections, leading to local privilege escalation to SYSTEM.

National Vulnerability Database (NVD)

- Understand the impact of the vulnerabilities
- Methodologies to mitigate the vulnerabilities
- CVSS Severity Rating
- Fix Information
- Vulnerable Software Versions
- SCAP Mappings
- CPE Information

The NVD logo is a large, stylized, dark blue graphic. It consists of the letters 'NVD' in a bold, sans-serif font. The 'N' and 'V' are connected, and the 'D' is a simple, rounded shape. The overall design is clean and modern.

- <https://nvd.nist.gov/vuln/detail/CVE-2026-7791>



VULNERABILITIES

CVE-2026-7791 Detail

RECEIVED

This CVE record has recently been published to the CVE List and has been included within the NVD dataset.

Description

Improper privilege management in the log rotation mechanism of the Skylight Workspace Config Service in Amazon WorkSpaces for Windows before 2.6.2034.0 allows a local non-admin authenticated user to place arbitrary files into arbitrary locations bypassing file system permission protections, leading to local privilege escalation to SYSTEM.

QUICK INFO

CVE Dictionary Entry:

[CVE-2026-7791](#)

NVD Published Date:

05/04/2026

NVD Last Modified:

05/04/2026

Source:

AMZN

Metrics

CVSS Version 4.0

CVSS Version 3.x

CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:



NIST: NVD

Base Score: N/A

NVD assessment not yet provided.

- The Common Attack Pattern Enumeration and Classification (CAPEC™) effort provides a publicly available catalog of common attack patterns that helps users understand how adversaries exploit weaknesses in applications and other cyber-enabled capabilities.
- Each attack pattern captures knowledge about how specific parts of an attack are designed, executed, and gives guidance on ways to mitigate the attack's effectiveness.
- Attack patterns help those developing applications or administering cyber-enabled capabilities to better understand the specific elements of an attack and how to stop them from succeeding.



CAPEC VIEW: Domains of Attack

View ID: 3000

Structure: Graph

▼ Objective

This view organizes attack patterns hierarchically based on the attack domain.

▼ Relationships

The following graph shows the tree-like relationships between attack patterns that exist at different levels of abstraction. At the common characteristic. Within categories, meta level attack patterns are used to present a decidedly abstract characterization c detailed level patterns that are focused on a specific methodology or technique used.

[Expand All](#) | [Collapse All](#)

3000 - Domains of Attack

- ☐  [Software - \(513\)](#)
 - ☐  [Exploitation of Trusted Credentials - \(21\)](#)
 - ☐  [Exploiting Trust in Client - \(22\)](#)
 -  [Forced Deadlock - \(25\)](#)
 - ☐  [Leveraging Race Conditions - \(26\)](#)
 -  [Fuzzing - \(28\)](#)
 - ☐  [Manipulating User State - \(74\)](#)
 - ☐  [Brute Force - \(112\)](#)
 - ☐  [API Manipulation - \(113\)](#)
 - ☐  [Authentication Abuse - \(114\)](#)
 - ☐  [Authentication Bypass - \(115\)](#)
 - ☐  [Excavation - \(116\)](#)
 - ☐  [Privilege Abuse - \(122\)](#)
 - ☐  [Buffer Manipulation - \(123\)](#)
 -  [Shared Data Manipulation - \(124\)](#)
 - ☐  [Flooding - \(125\)](#)
 -  [Pointer Manipulation - \(129\)](#)
 - ☐  [Excessive Allocation - \(130\)](#)
 -  [Resource Leak Exposure - \(131\)](#)
 - ☐  [Parameter Injection - \(137\)](#)
 - ☐  [Content Spoofing - \(148\)](#)
 - ☐  [Identity Spoofing - \(151\)](#)
 - ☐  [Input Data Manipulation - \(152\)](#)

- MITRE ATT&CK[®] is a globally-accessible knowledge base of adversary tactics and techniques based on real-world observations.

ATT&CK[®]

Attack Categories and Tactics

MITRE | ATT&CK®

Matrices ▾

Tactics ▾

Techniques ▾

Defenses ▾

CTI ▾

Resources ▾

Benefactors

Contribute

Blog ↗

TACTICS

Enterprise



Reconnaissance

Resource Development

Initial Access

Execution

Persistence

Privilege Escalation

Stealth

Defense Impairment

Credential Access

Discovery

Lateral Movement

Collection

Command and Control

Exfiltration

Impact

Enterprise tactics

Tactics represent the "why" of an ATT&CK technique or sub-technique. It is the adversary's tactical goal: the reason for performing an action. For example, an adversary may want to achieve credential access.

Enterprise Tactics:
15

ID	Name	Description
TA0043	Reconnaissance	The adversary is trying to gather information they can use to plan future operations.
TA0042	Resource Development	The adversary is trying to establish resources they can use to support operations.
TA0001	Initial Access	The adversary is trying to get into your network.
TA0002	Execution	The adversary is trying to run malicious code.
TA0003	Persistence	The adversary is trying to maintain their foothold.
TA0004	Privilege Escalation	The adversary is trying to gain higher-level permissions.
TA0005	Stealth	The adversary is trying to hide and conceal their actions, appearing as normal behavior.
TA0112	Defense Impairment	The adversary is trying to break security mechanisms, pipelines, and tooling so defenders can't see or trust what's happening.

Mitigation Techniques

MITRE | ATT&CK®

Matrices ▾

Tactics ▾

Techniques ▾

Defenses ▾

CTI ▾

Resources ▾

Benefactors

Contribute

Blog ↗

Se

MITIGATIONS

Antivirus/Antimalware

**Application Developer
Guidance**Application Isolation and
Sandboxing

Audit

Behavior Prevention on
Endpoint

Boot Integrity

Code Signing

Credential Access
Protection

Data Backup

Data Loss Prevention

Disable or Remove Feature
or Program

Do Not Mitigate

Encrypt Sensitive
InformationEnvironment Variable
Permissions

Techniques Addressed by Mitigation

ATT&CK® Navigator Layers ▾

Domain	ID	Name	Use
Enterprise	T1212	Exploitation for Credential Access	Application developers should consider taking measures to validate authentication requests by enabling one-time passwords, providing timestamps or sequence numbers for messages sent, using digital signatures, and/or using random session keys. ^{[1][2]}
Enterprise	T1564	Hide Artifacts	Application developers should consider limiting the requirements for custom or otherwise difficult to manage file/folder exclusions. Where possible, install applications to trusted system folder paths that are already protected by restricted file and directory permissions.
	.009	Resource Forking	Configure applications to use the application bundle structure which leverages the <code>/Resources</code> folder location. ^[3]
	.012	File/Path Exclusions	Application developers should consider limiting the requirements for custom or otherwise difficult to manage file/folder exclusions. Where possible, install applications to trusted system folder paths that are already protected by restricted file and directory permissions.
Enterprise	T1574	Hijack Execution Flow	When possible, include hash values in manifest files to help prevent side-loading of malicious libraries. ^[4]
	.001	DLL	When possible, include hash values in manifest files to help prevent side-loading of malicious libraries.
Enterprise	T1559	Inter-Process Communication	Enable the Hardened Runtime capability when developing applications. Do not include the <code>com.apple.security.get-task-allow</code> entitlement with the value set to any variation of true.

SOFTWARE ASSURANCE METHODOLOGIES



Software assurance methodologies

Three “standard” models :

1. Building Security In Maturity Model
2. Open Software Assurance Maturity Model
3. Microsoft secure software development life cycle



Building Security In Maturity Model

- BSIMM is a *Maturity Model* for real-world best practices in software-producing companies, started in 2008.
 - examines Software Security Initiatives (SSIs)
 - provides a “measuring stick”, state-of-the-art
- Introduced by Gary McGraw and others
 - Author of Software Security: Building Security In Maturity Mode
- Inspired by *Capability Maturity Model* (CMM) (late 80s-90s)
 - model of software development processes
 - maturity = degree of formality/rigor of process
 - 5 Levels: chaotic, repeatable, defined, managed, optimizing
- Now at BSIMM-16, Jan. 2026, with 128 activities, 8 industries, and 111 organizations
 - The report provides insights into real-world application security practices protecting approximately 91,200 applications developed by 223,700 developers



BSIMM goals

For organizations starting/running a Software Security Initiative, BSIMM aims to:

- Inform risk management decisions
- Clarify “right things to do” for those involved
- Reduce costs via standard, repeatable processes
- Improve code quality

This is done by planning a **Software Security Initiative**, implementing activities selected from BSIMM. Activities can be rolled out according to the maturity level of the organization.

BSIMM Software Security Framework

- BSIMM defines a Software Security Framework which describes
 - 12 practices organized into 4 domains
 - Governance, Intelligence, Development (SSDL Touchpoints), Deployment
 - Each practice involves numerous activities
 - Each practice split into maturity levels 1–3
 - each maturity level has several activities
- Now covers over 128 activities
- New activities added when they appear in more than 1 organization

BSIMM16 key trends and insights

- **AI is now the defining challenge in application security.** Organizations are simultaneously securing AI-powered coding assistants and defending against AI-enabled attacks.
- **Government regulations are accelerating major security investments.** Global mandates are pushing organizations to strengthen application security, with a sharp focus on software supply chain transparency and securing development environments.
- **Software supply chain security is rapidly rising in importance.** Organizations are expanding their focus beyond internally developed code to secure the entire software supply chain ecosystem.
- **Application security training is undergoing a major shift.** Traditional multi-day security courses are being replaced by just-in-time, bite-sized learning that fits modern development workflows and learner preferences.

BSIMM domains and practices

Governance



Practices that help you organize, manage and measure your software security initiative including staff development.

Strategy & Metrics

Compliance & Policy

Training

Intelligence



Practices that result in collections of knowledge to use to carry out software security activities throughout your organization.

Attack Models

Security Features & Design

Standards & Requirements

SSDL Touchpoints



Common practices associated with analysis and assurance of particular software development artifacts and processes.

Architecture Analysis

Code Review

Security Testing

Deployment



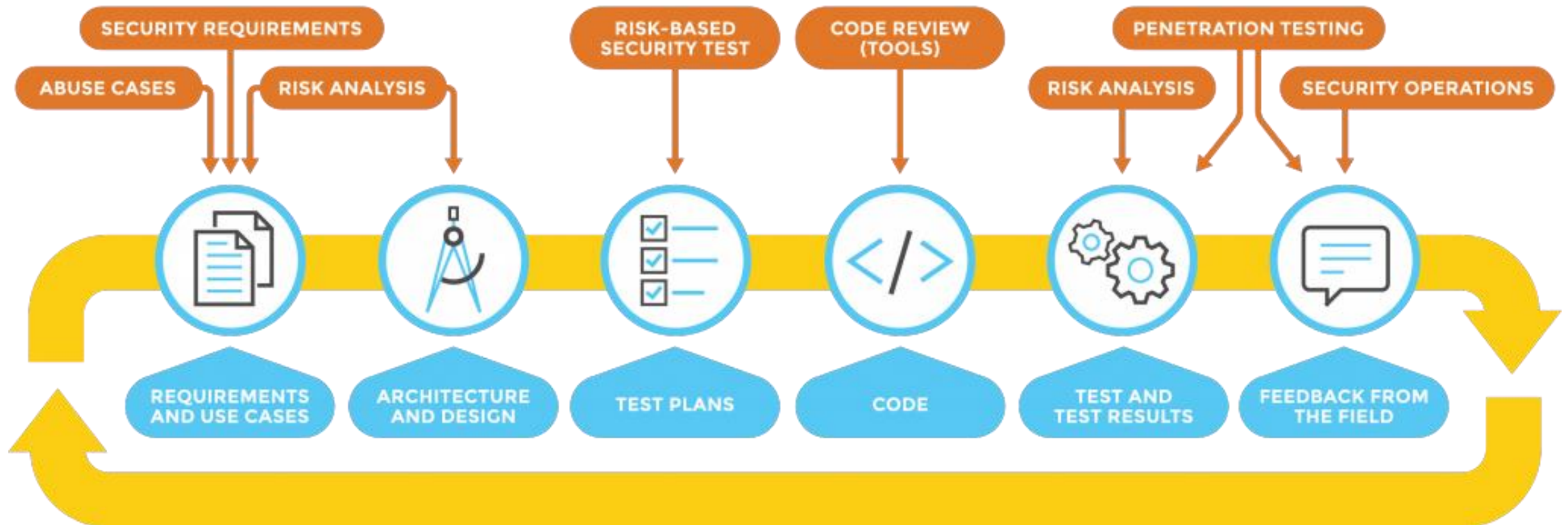
Practices that interface with traditional network security and software maintenance organizations.

Penetration Testing

Software Environment

Config Management & Vulnerability Management

SSDL Touchpoints



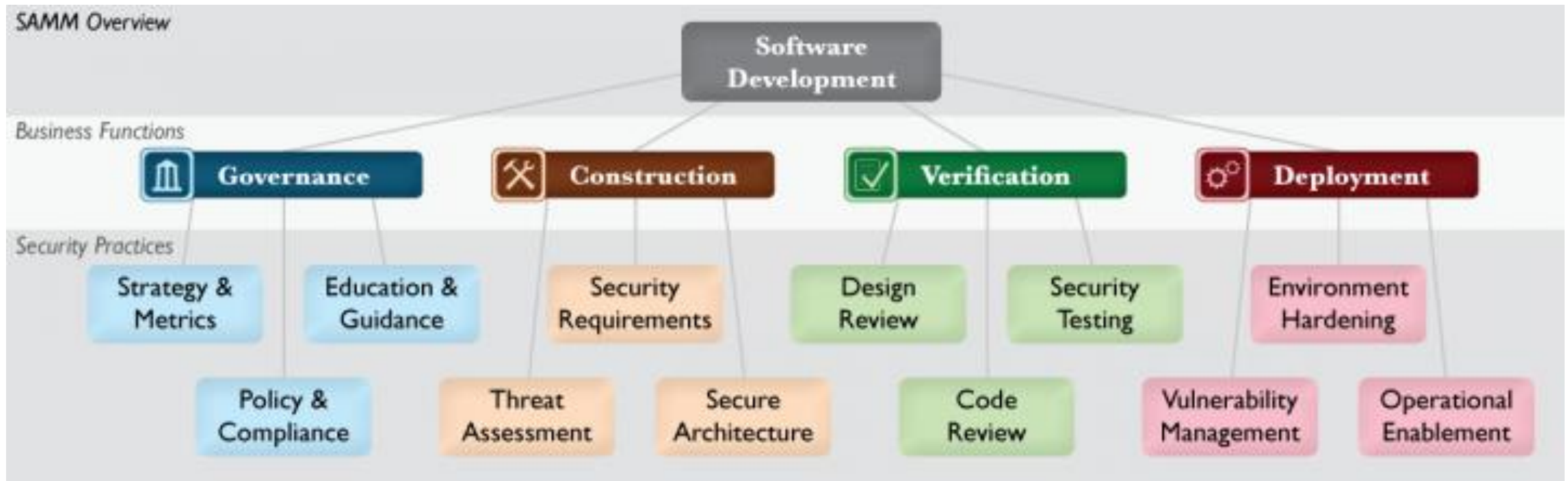
Open Software Assurance Maturity Model

- The Software Assurance Maturity Model (SAMM) is an open framework to help organizations formulate and implement a strategy for software security that is tailored to the specific risks facing the organization.
- Also, under the OWASP project



Open Software Assurance Maturity Model

- From each of the Business Functions, 3 Security Practices are defined
- The Security Practices cover *all* areas relevant to software security assurance

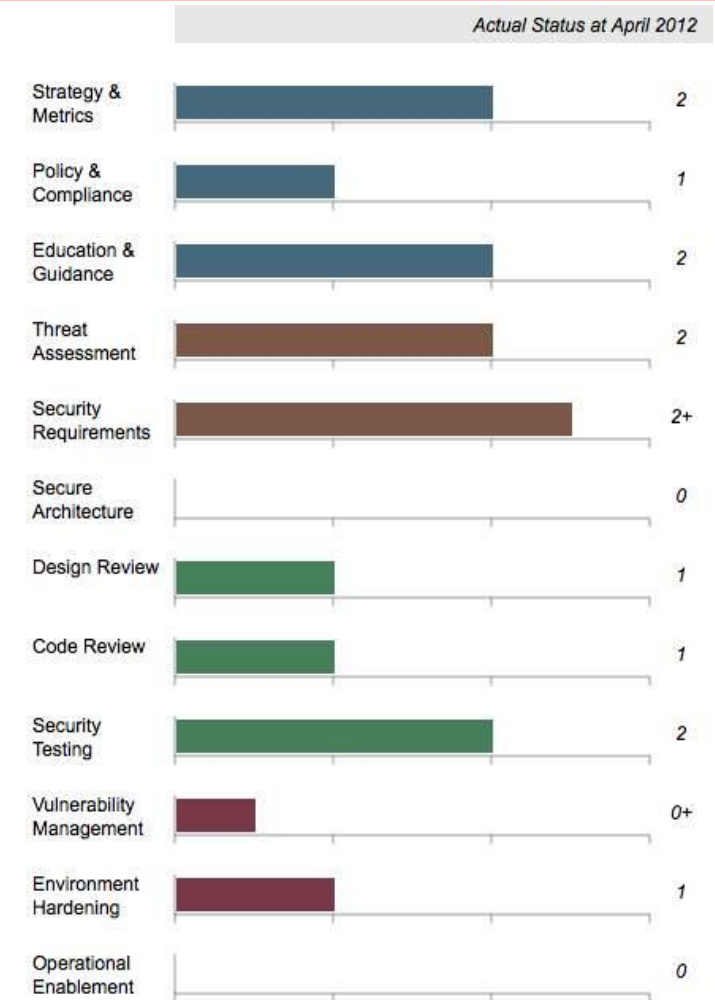


Under each security practice

- Three successive Objectives under each Practice define how it can be improved over time
- This establishes a notion of a Level at which an organization fulfills a given Practice
- The three Levels for a Practice generally correspond to:
 - 0: Implicit starting point with the Practice unfulfilled
 - 1: **Initial** understanding and ad hoc provision of the Practice
 - 2: **Increase** efficiency and/or effectiveness of the Practice
 - 3: **Comprehensive** mastery of the Practice at scale

Creating Scorecards

- Gap analysis
 - Capturing scores from detailed assessments versus expected performance levels
- Demonstrating improvement
 - Capturing scores from before and after an iteration of assurance program build-out



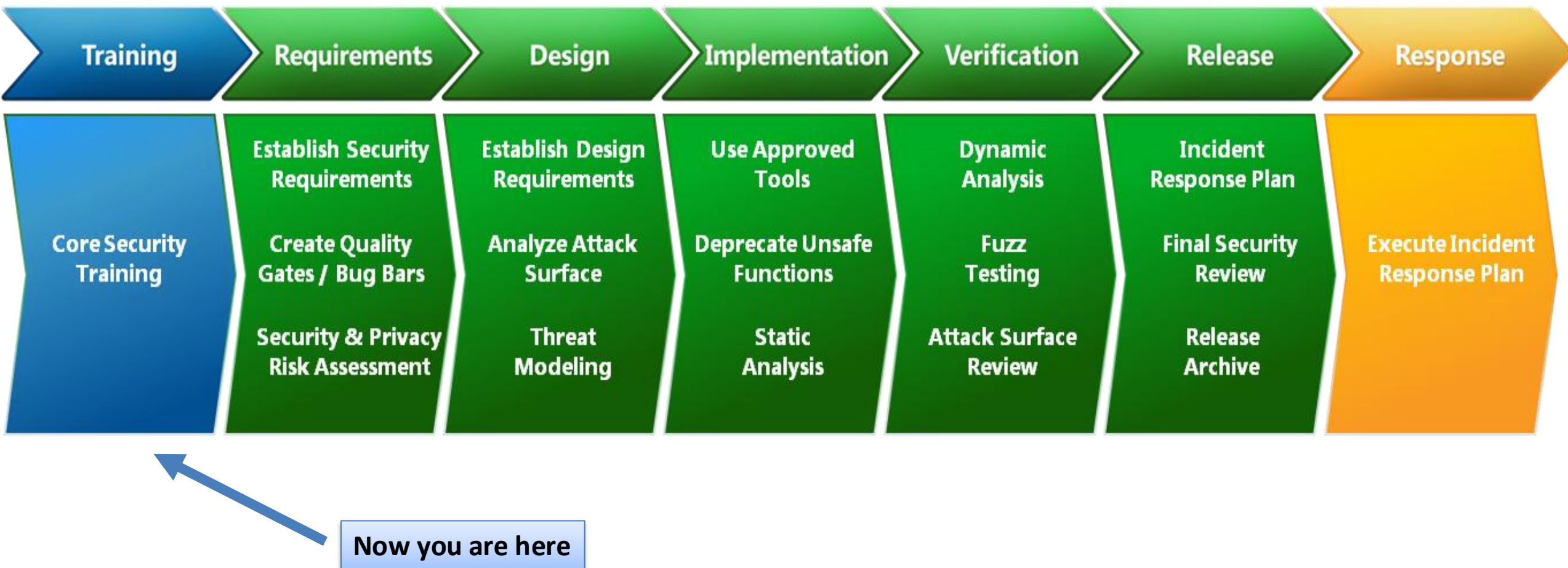
Example Organization Scorecard

Microsoft secure software development life cycle

- The **Microsoft Security Development Lifecycle (SDL)** was an outcome of Microsoft software development groups working to develop **a security model** that's easy for developers to understand and build into their security code.
- Introduces security and privacy considerations throughout all phases of the development process, helping developers **build highly secure software**, address security compliance requirements, and reduce development costs.

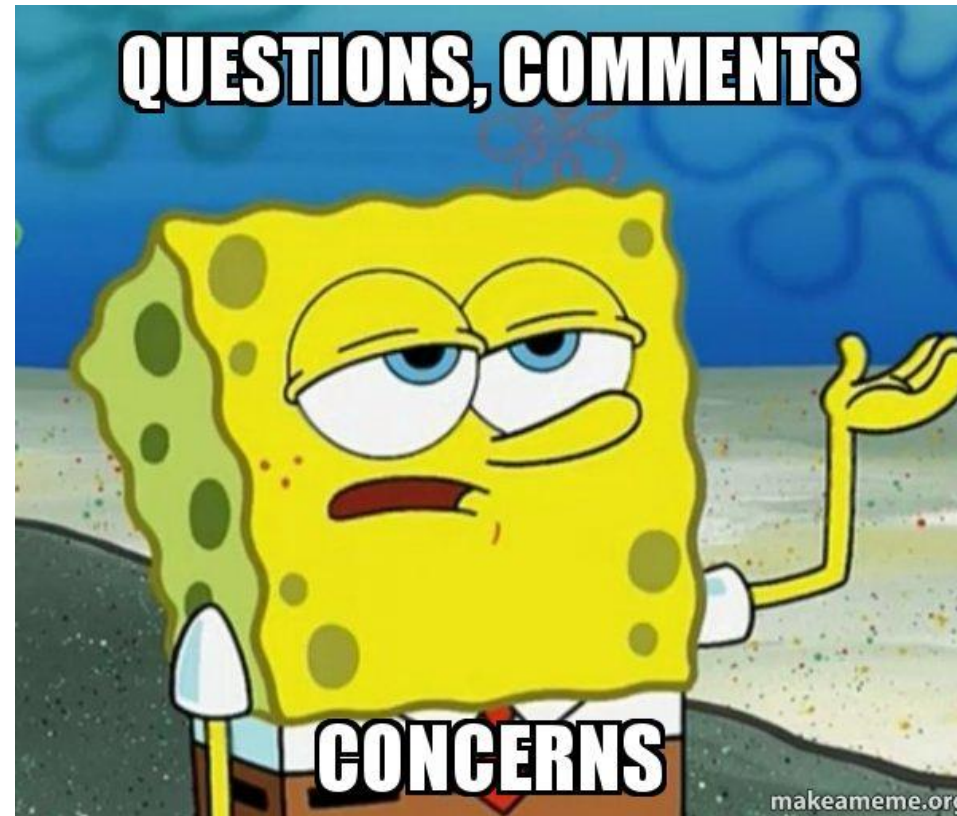


Microsoft secure software development life cycle



Summary

1. Relationship between Software and Security
2. Software Vulnerabilities:
 - OWASP top 10
 - CVEs
 - NVD
 - CAPEC
 - MITRE ATT&CK
3. Software Assurance Methodologies
 - BSIMM
 - OpenSAMM
 - Microsoft SDLC



QUESTION AND COMMENTS?